ETH zürich

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

NextRAN-AI – 07/11/2025

Integrated Systems Laboratory (ETH Zürich)

Marco Bertuletti mbertuletti@iis.ee.ethz.ch
Yichao Zhang yiczhang@iis.ee.ethz.ch
Mahdi Abdollahpour mahdi.abdollahpour@unibo.it
Alessandro Vanelli-Coralli avanelli@iis.ee.ethz.ch
Luca Benini lbenini@iis.ee.ethz.ch

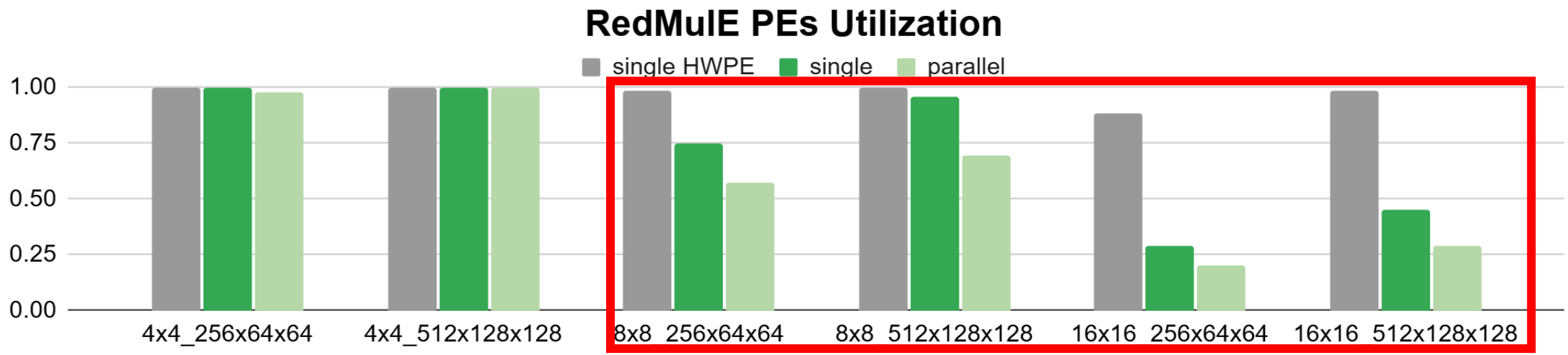
PULP Platform
Open Source Hardware, the way it should be!

pulp-platform.org
@pulp_platform
company/pulp-platform
youtube.com/pulp_platform

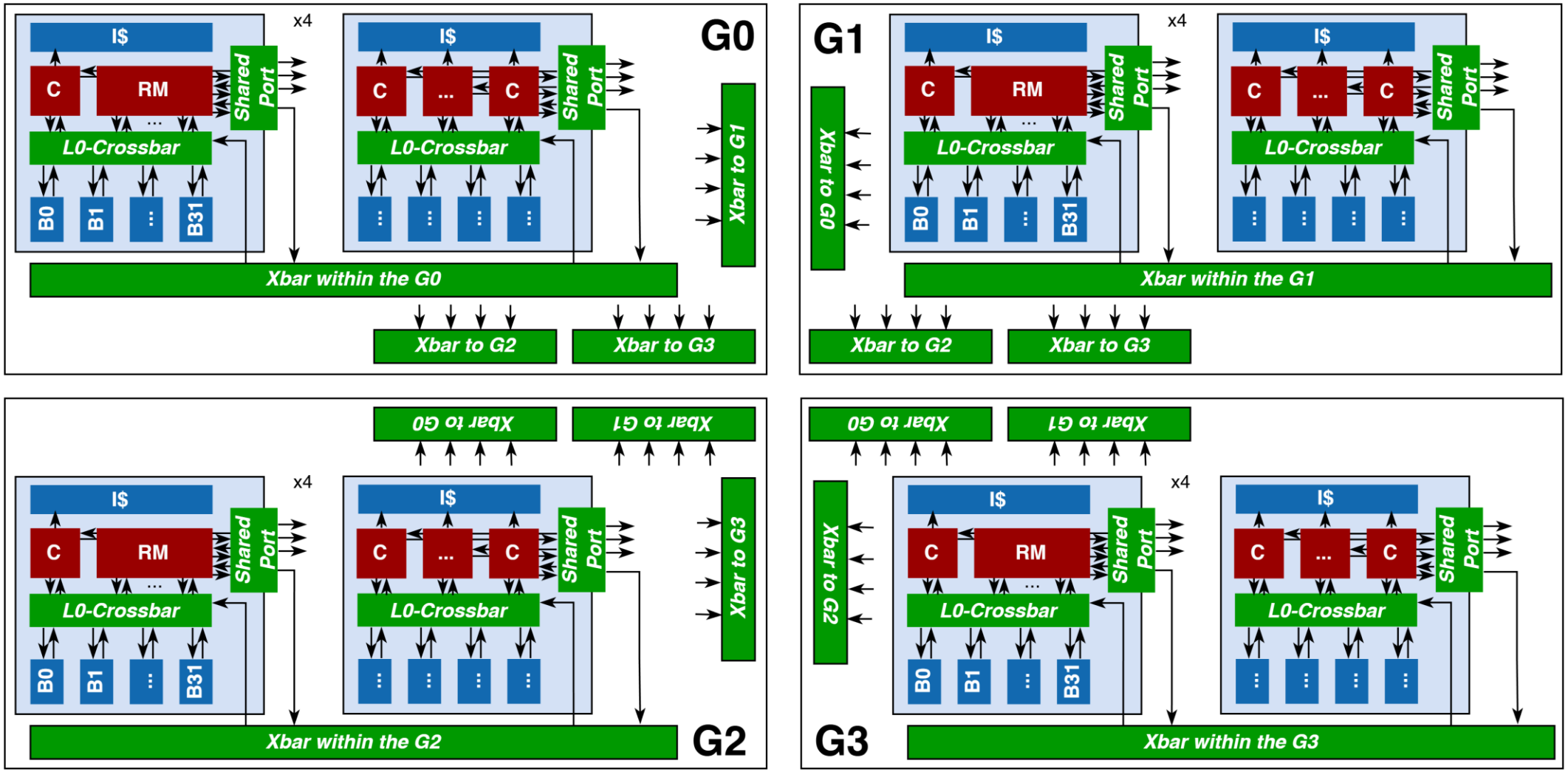
Outline

- Improvement of performance on large RedMulE configurations
- Results on parallelization for large RedMulE configurations

Parallel 16x16 has low utilization…

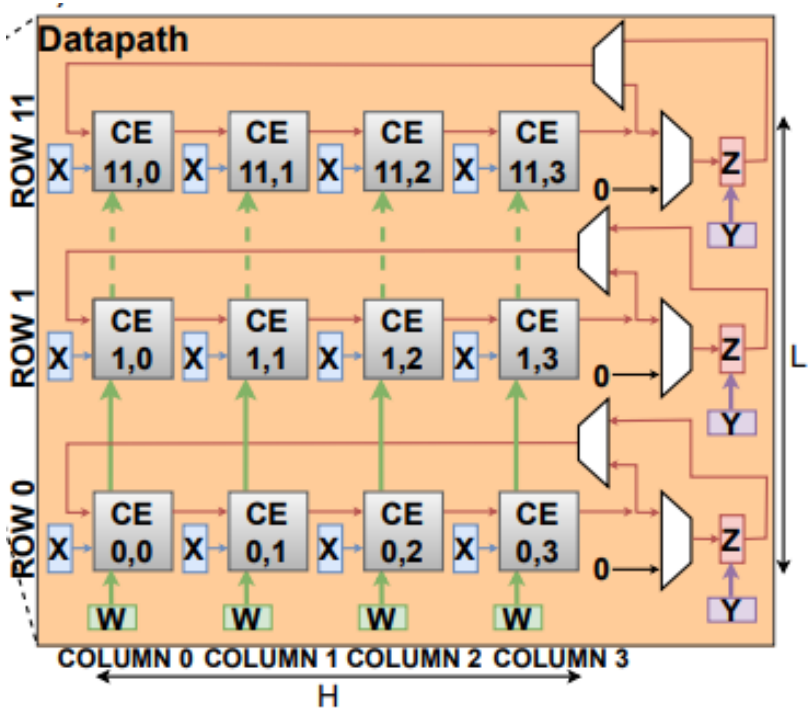


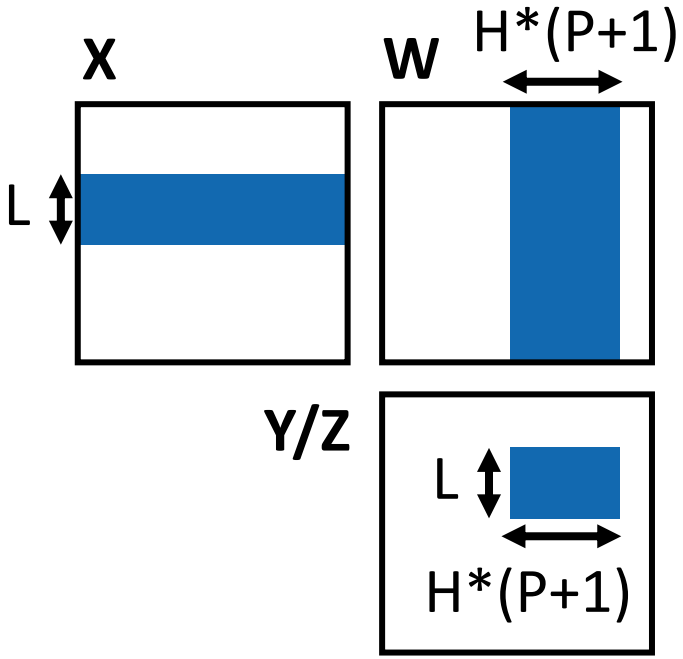
TensorPool-64 (4 RedMulEs)



Single RedMulE low Util., Z is the bottleneck…

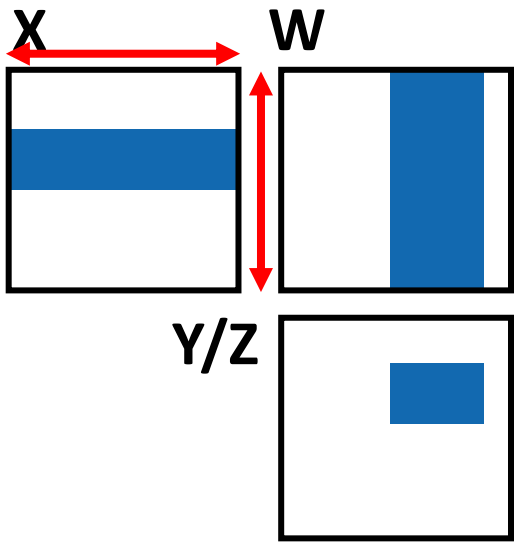
- Accelerator stalls on writing out the Z values:
  - Zs and Ys share the same buffer,
  - at the end of the computation we must write out the Zs,
  - if Z is not written I cannot load new Ys.

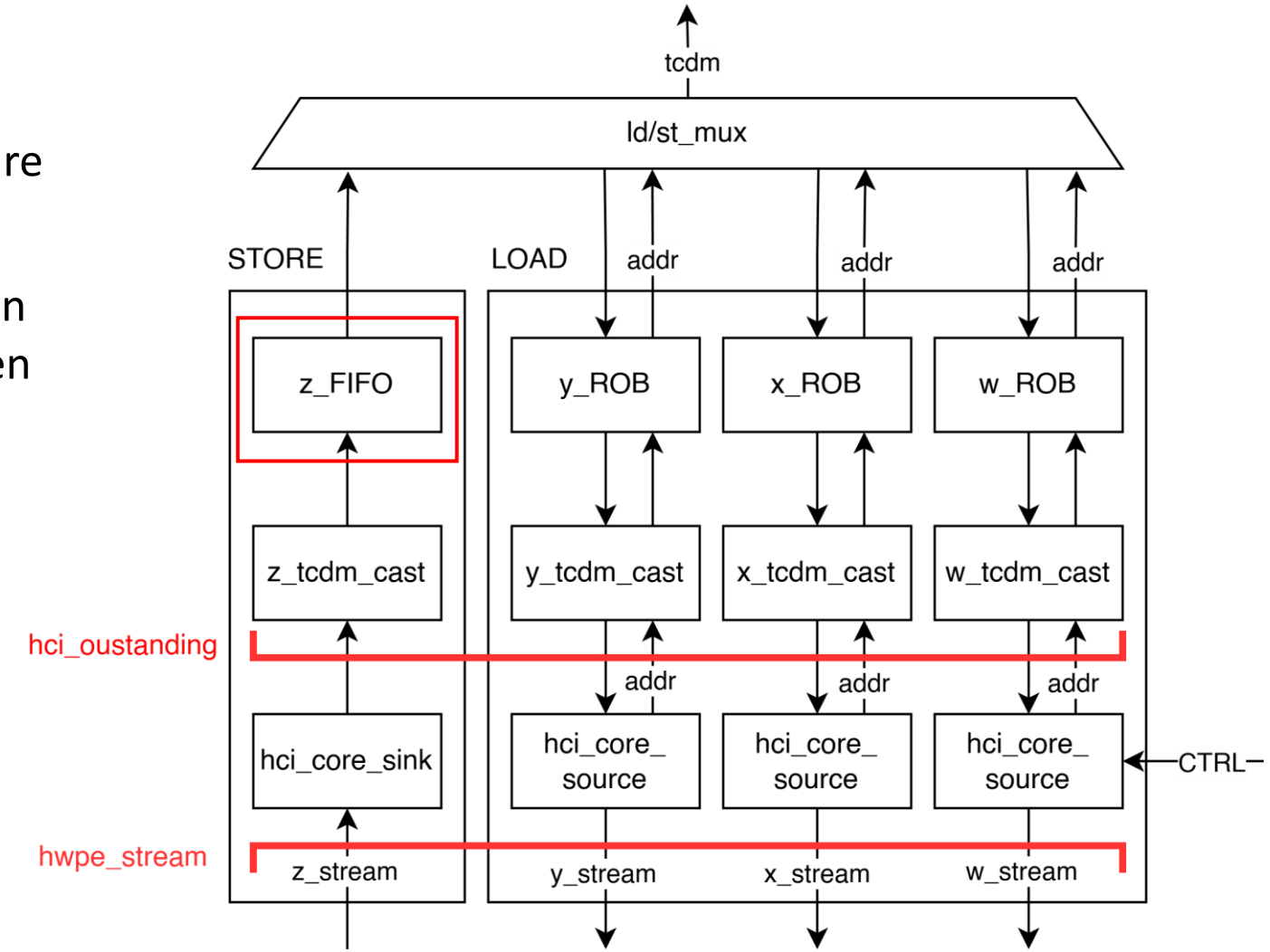




How to optimize the write-out of Z?

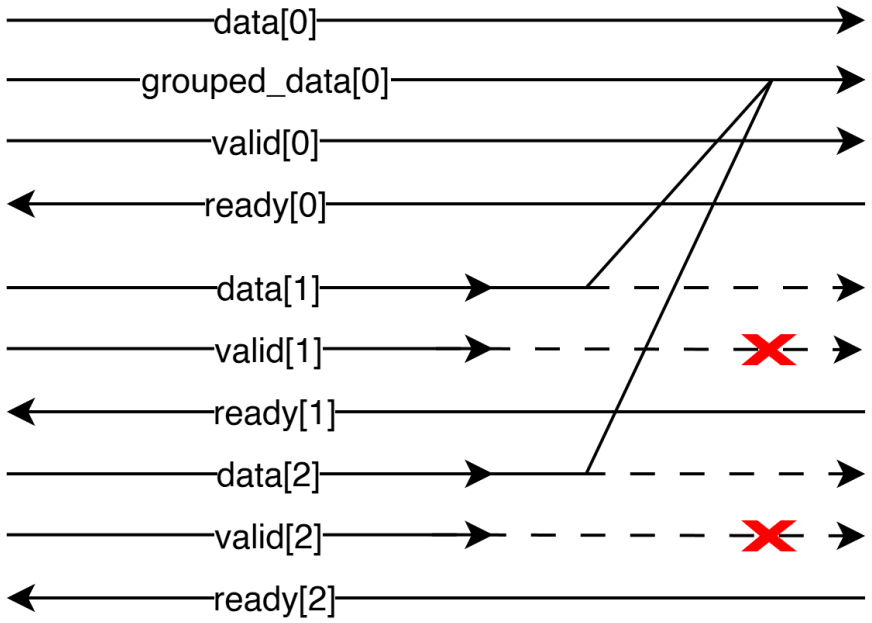
- Add a buffer for Z values
  - It can be a FIFO, we do not care about the order of Z writes
  - If the inner product dimension is large, Z writes can be hidden



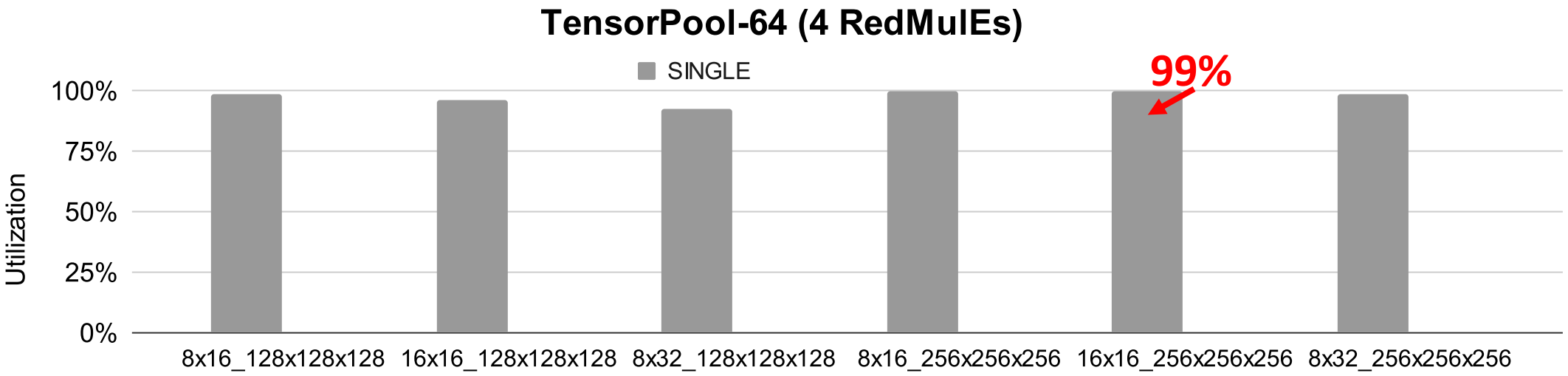


How to optimize the write-out of Z?

- Group write-requests under the same valid/ready handshake
  - Increases the bandwidth across all the hierarchy levels
  - Is parametrizable (2x, 4x, 8x, depending on the available routing resources)



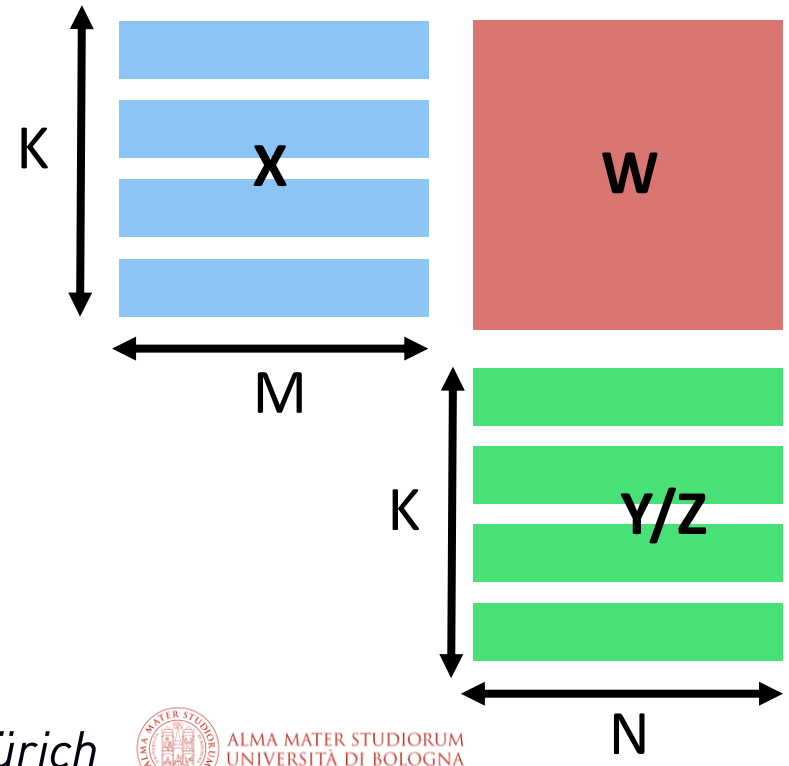
Single Core has high utilization



Parallelization

Row-wise parallelization:

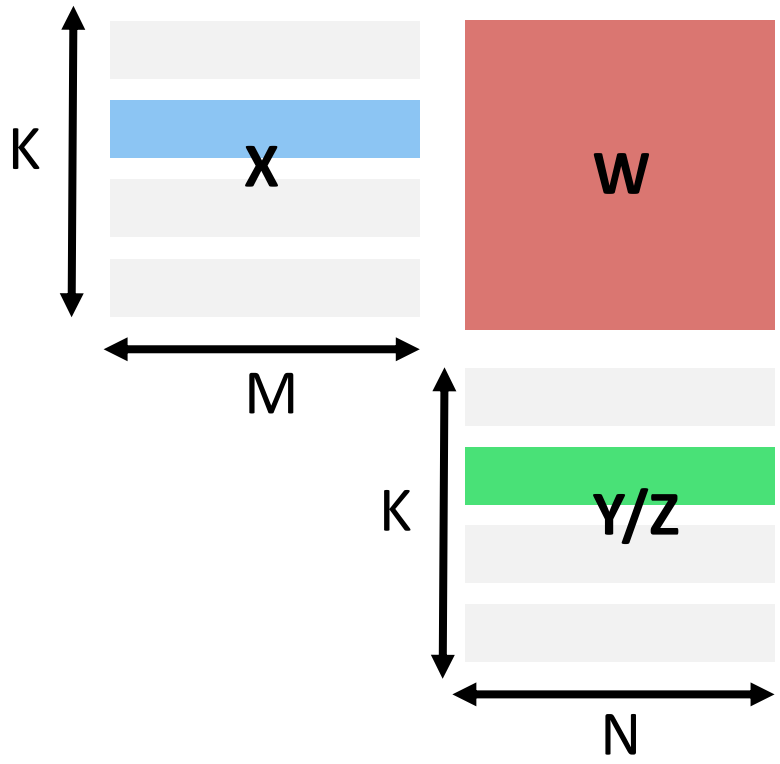
- Each RedMulE gets a set of rows in X
- Each RedMulE gets a set of rows in Y
- W is shared




```
// Matmul, MxKxN, all data in L1

uint32_t redmule_id = get_redmule_id();
uint32_t redmule_num = get_redmule_num();

for (i = 0; i < num_redmules; i++) {
  redmule_cfg (
    ptr_X + redmule_id * M * K / num_redmules,
    ptr_W,
    ptr_Y + redmule_id * M * K / num_redmules,
    M, K / num_redmules, N,
    GEMM, Float16);
    hwpe_trigger_job();
}
mempool_barrier();
```

Parallelization

Each RedMulE executes an M*(K/num_redmules)*N **GEMM**



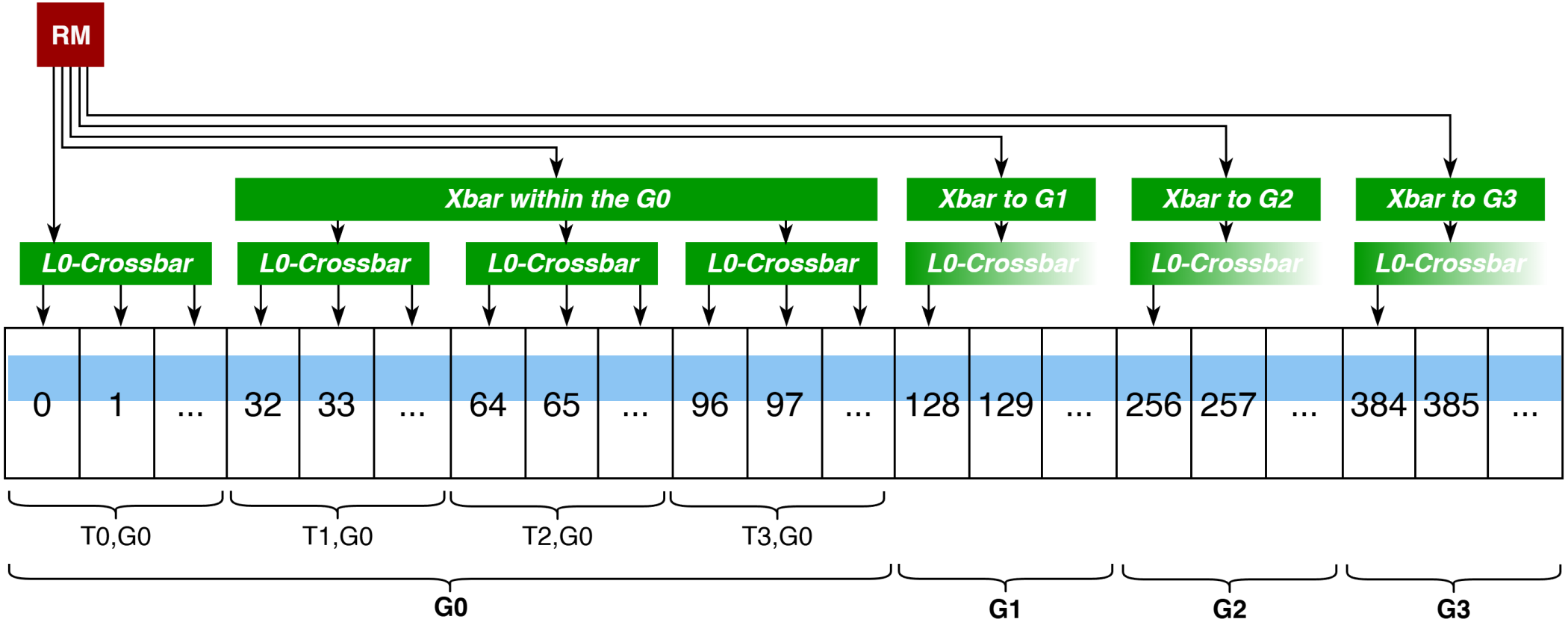
Contentions on X and W!

```
// Matmul, MxKxN, all data in L1

uint32_t redmule_id = get_redmule_id();
uint32_t num_redmules = get_redmule_num();

for (i = 0; i < num_redmules; i++) {
  redmule_cfg (
    ptr_X + redmule_id * M * K / num_redmules,
    ptr_W,
    ptr_Y + redmule_id * M * K / num_redmules,
    M, K / num_redmules, N,
    GEMM, Float16);
    hwpe_trigger_job();
}
mempool_barrier();
```

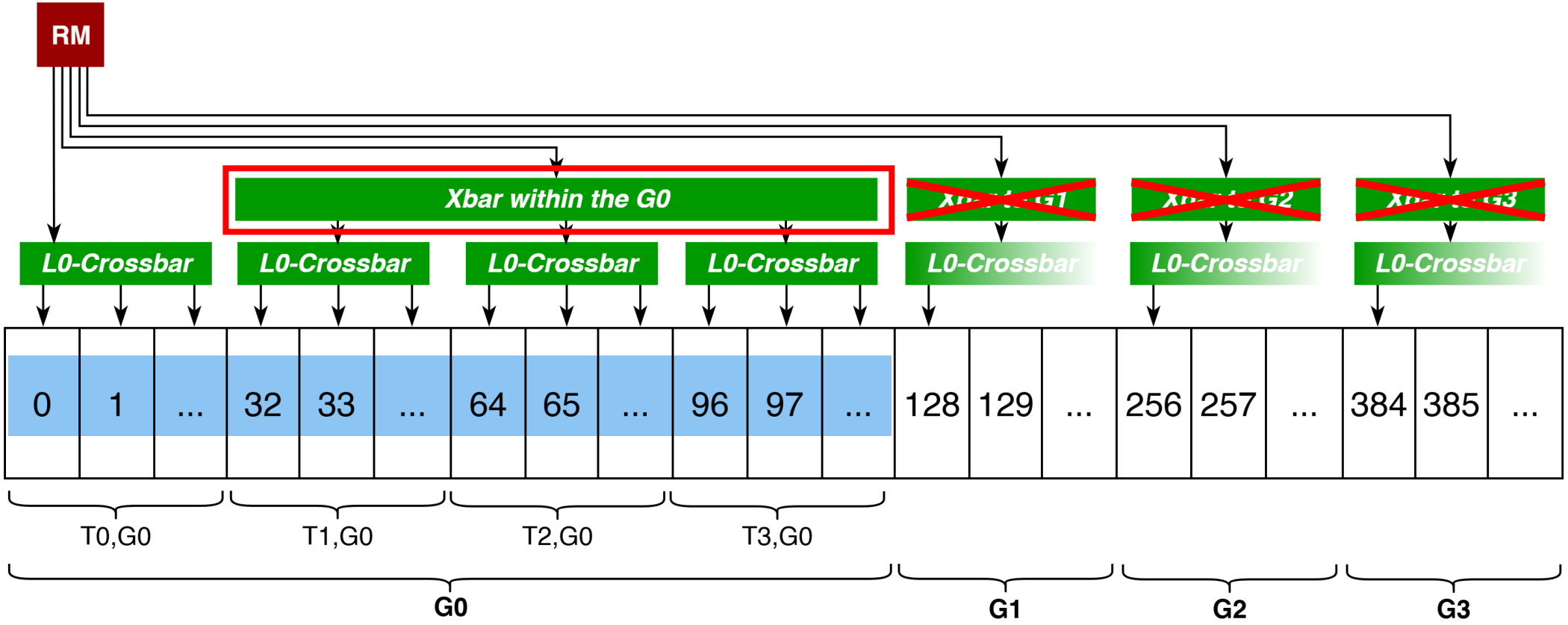
How to reduce the contentions on X?

- Each RedMulE accesses independent portions of the X matrix.
- However the data might be in the same bank → **bank contentions**
- Solution in memory layout:
  1. Allocate the portions of X in banks where they will only accessed by one RedMulE
  2. Shift the starting point of the X portions over banks

1. Allocate Locally (in the banks of a Group)



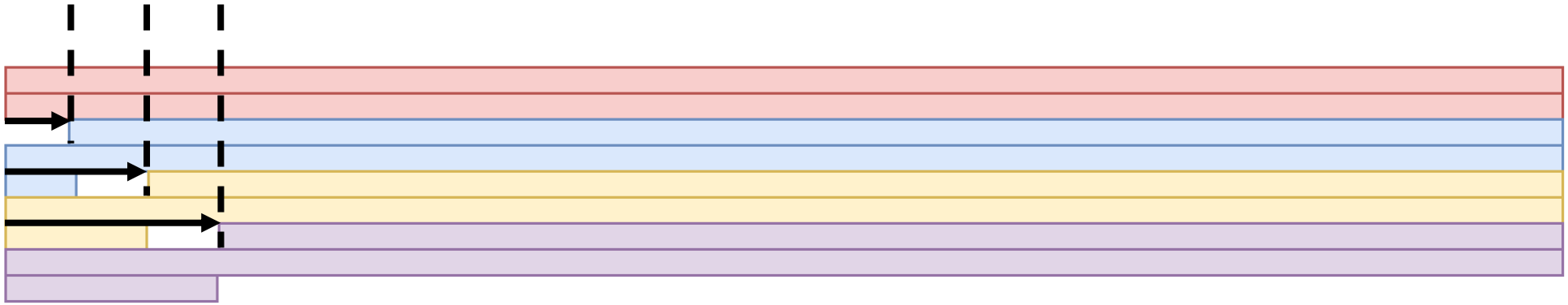
1. Allocate Locally (in the banks of a Group)



- Each RedMulE only uses one crossbar
- Therefore the BW is reduced → we observe a **performance drop**

2. Allocate in banks with a shift

- Apply a shift to the starting point of each portion of X
- RedMulEs will use all of the interconnection resources
- They will not access the same X banks at once



How to reduce the contentions on W?

Make sure that RedMulEs do not access the same portion of W at once

```
for i:m:M
  for j:n:N
    load (Y)(m × n)
    for l:k:K
      load (X)(m × k)
      load (W)(k × n)
      Y += X × W
    end
    store (Y)(m × n)
  end
end
```

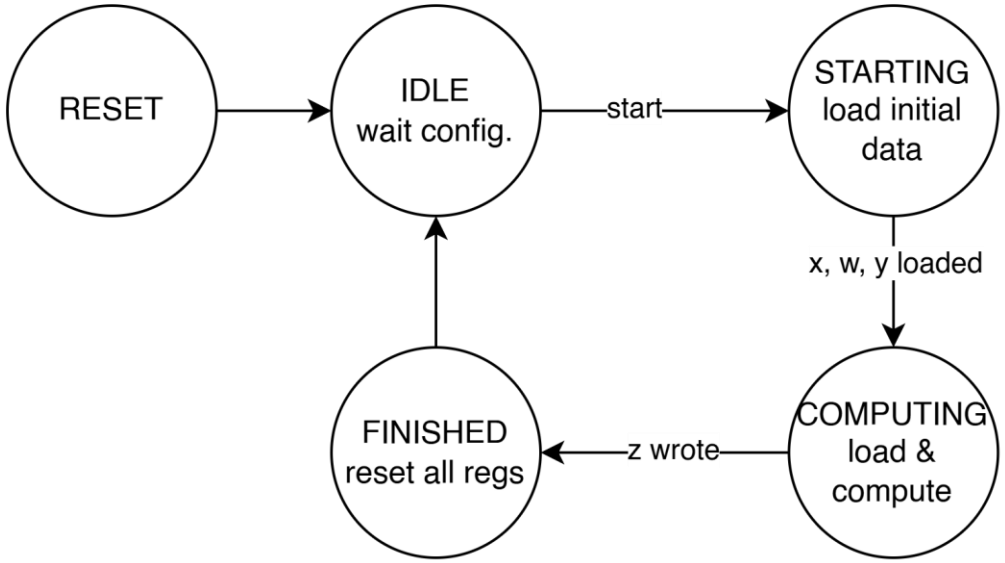
```
for i = p*M/4:m:(p+1)*M/4
  for j:n:N
    load (Y)(m × n)
    for l:k:K
      load (X)(m × k)
      load (W)(k × n)
      Y += X × W
    end
    store (Y)(m × n)
  end
end
```

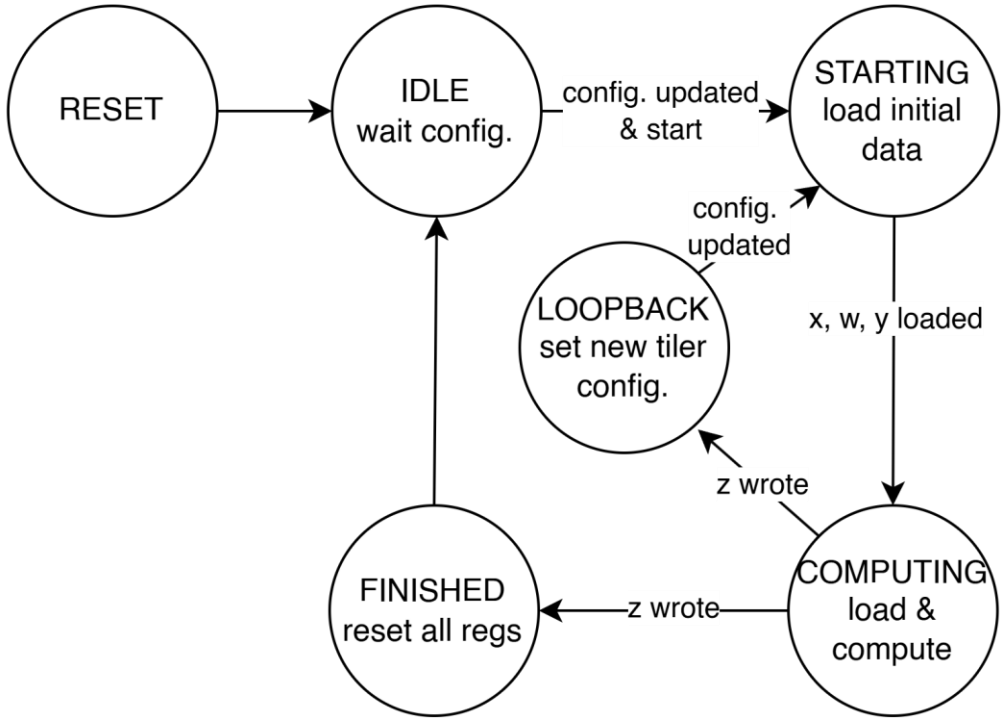
```
for i = p*M/4:m:(p+1)*M/4
  for j = offset:n:N
    load (Y)(m × n)
    for l:k:K
      load (X)(m × k)
      load (W)(k × n)
      Y += X × W
    end
    store (Y)(m × n)
  end
  for j = 0:n:offset
    load (Y)(m × n)
    for l:k:K
      load (X)(m × k)
      load (W)(k × n)
      Y += X × W
    end
    store (Y)(m × n)
  end
end
```

We can add an offset on the starting point over W columns and then loopback.

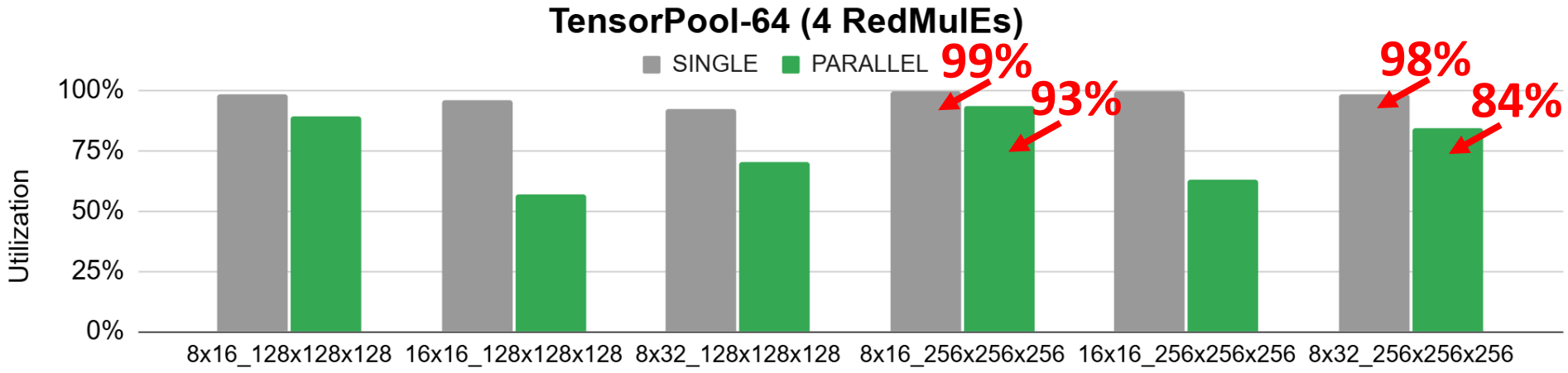
Modifications to RedMulE for Offset on W

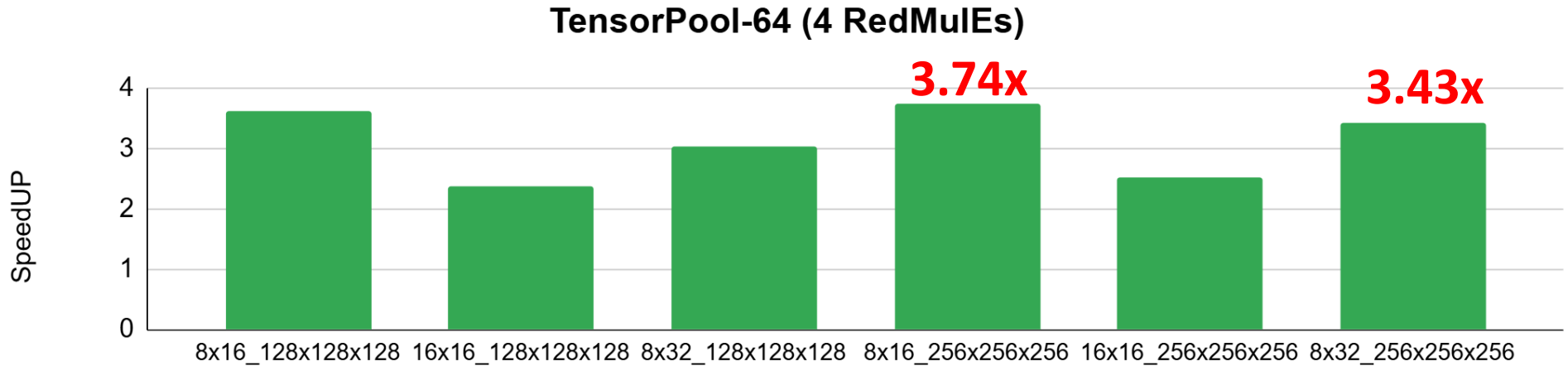
- Added register to specify the offset
- Modified the state machine to loopback



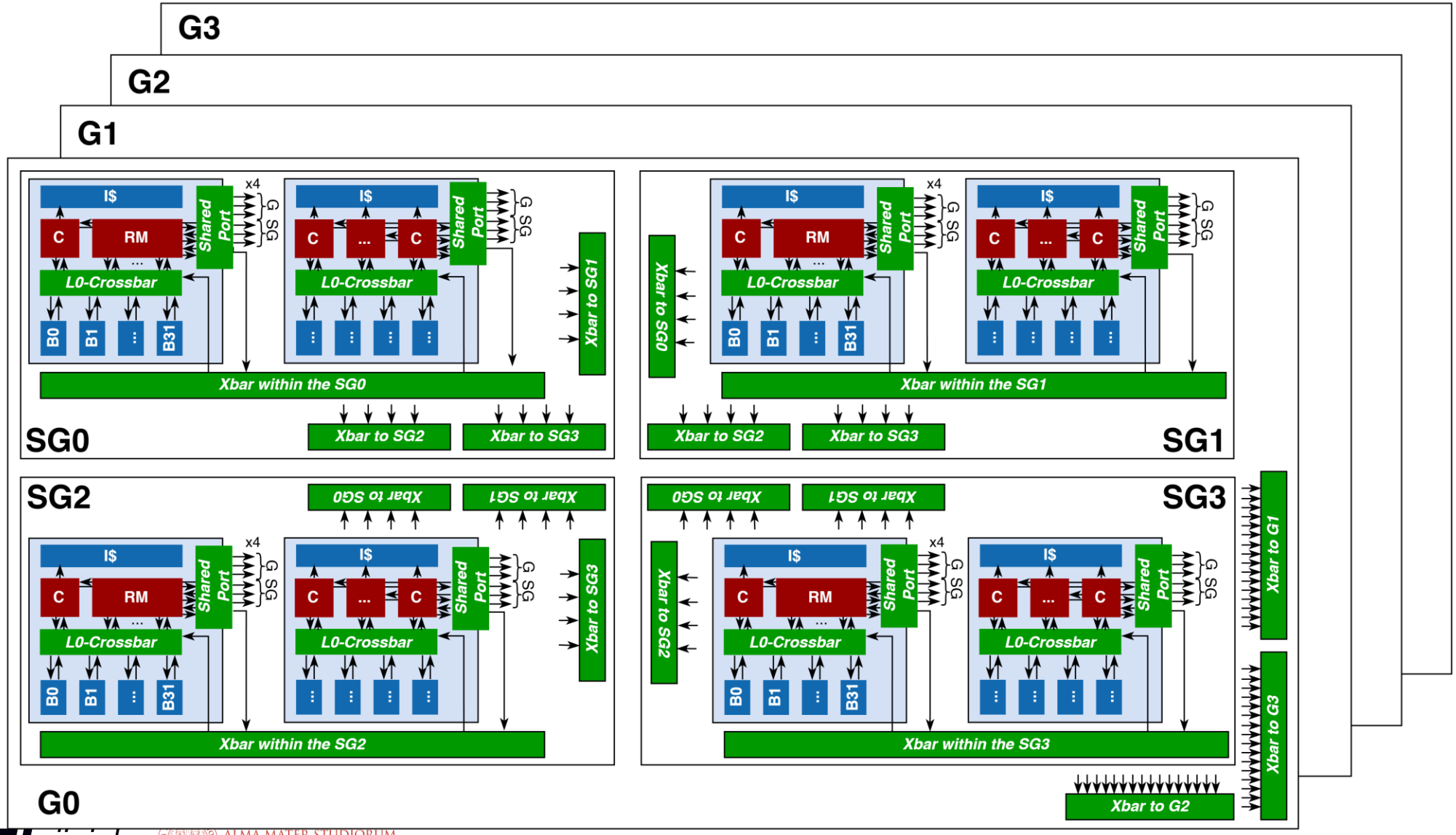


Improvement in utilization & SpeedUp

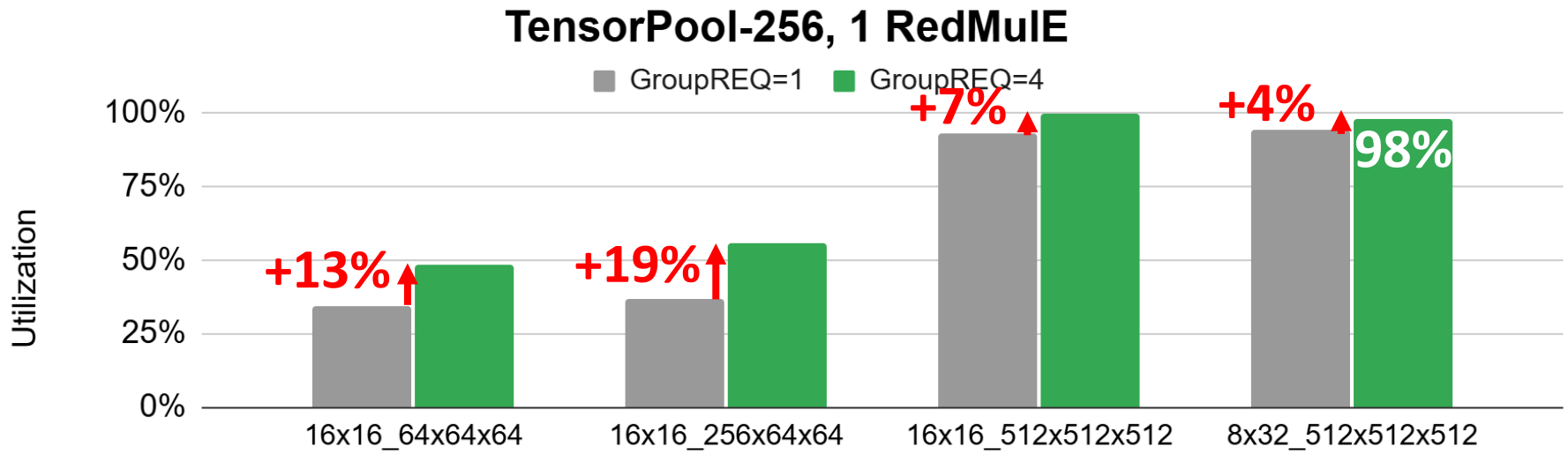


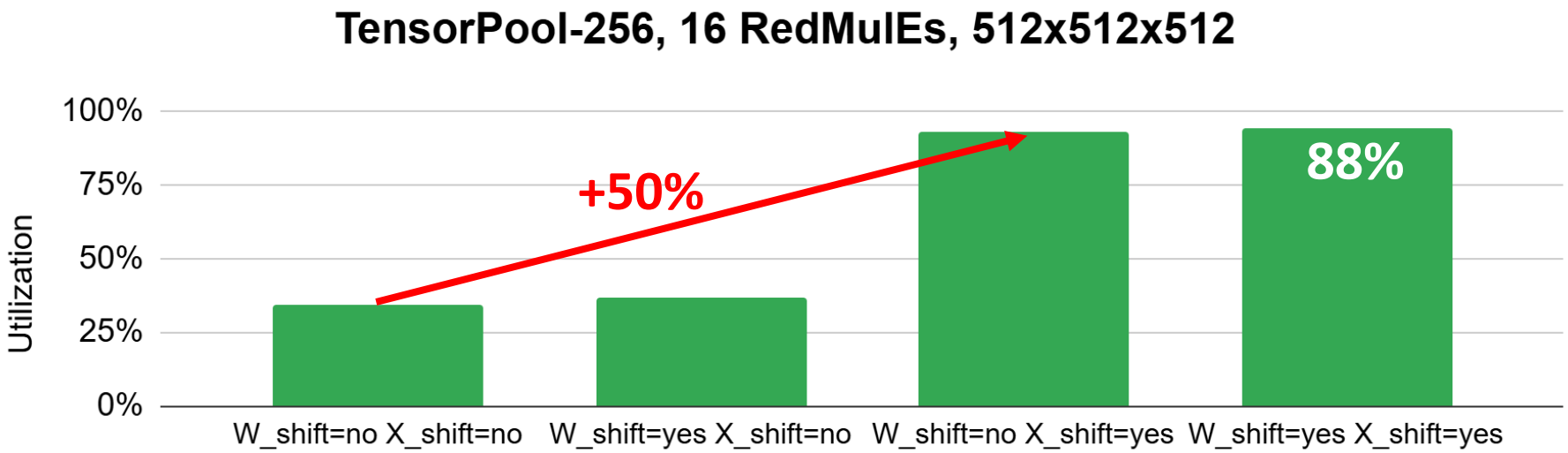


TensorPool-256 (16 RedMulEs, 4096 MAC / cycle)

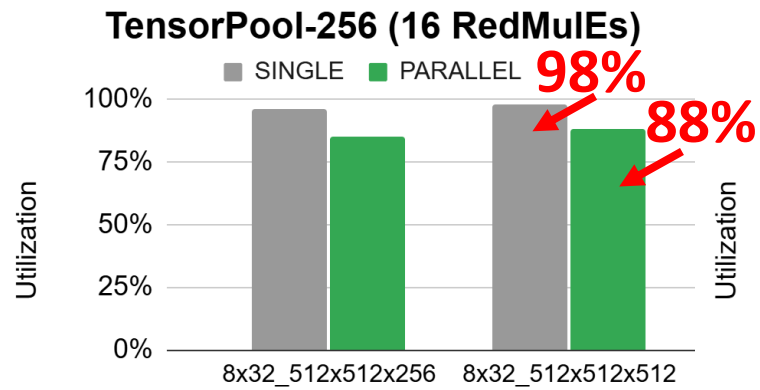


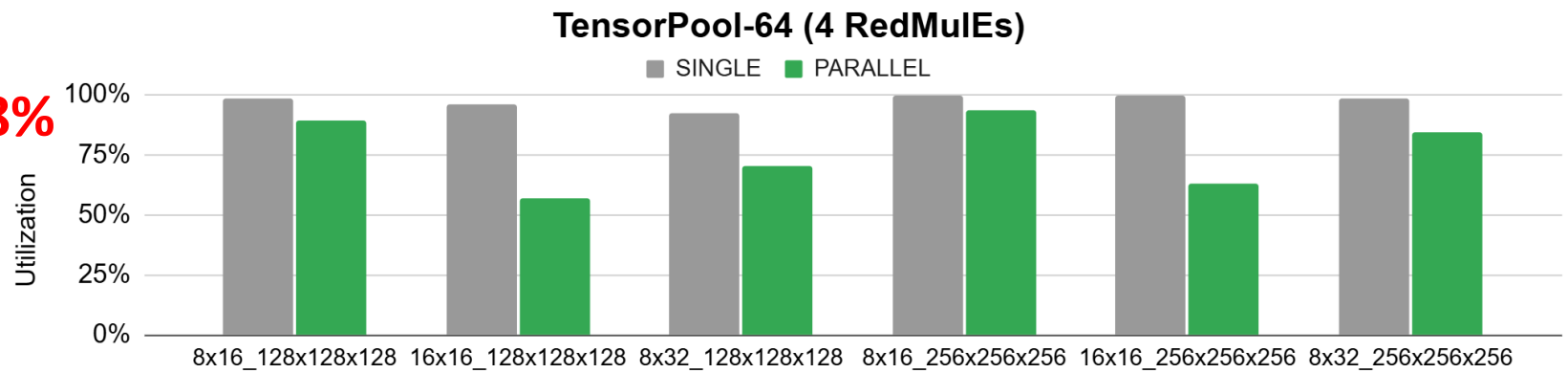
Utlization & Speed-Up

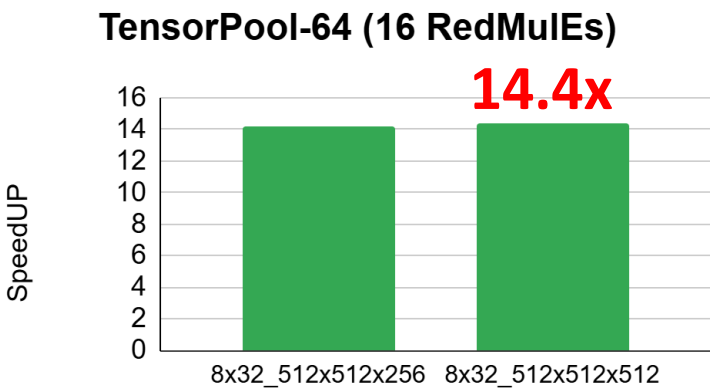


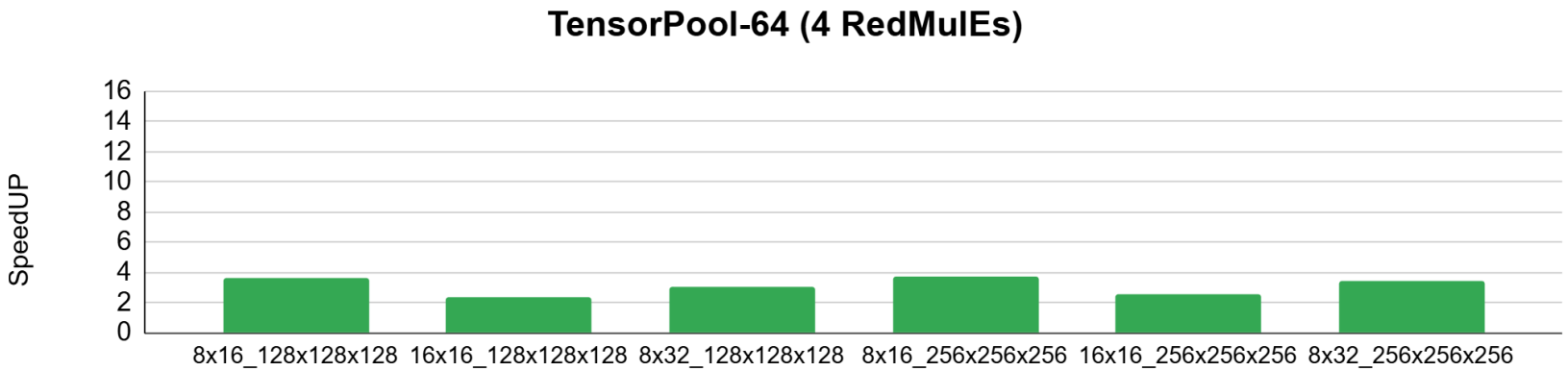


Utlization & Speed-Up









Next Steps

- **TensorPool:**
  - TeraPool + RedMulE (TensorPool) physical design in 7nm
  - PPA on model microkernels and operators (combined RedMulE&Cores)
- **System Performance:**
  - Simulation speed is impaired by large design size → from RTL simulation to higher abstraction level, e.g. GVSoC
  - TensorPool GVSoC model developement
  - Data-Movement and end-end performance